

EXACT-Coding TDD Experiment: Vollstaendiger Ueberblick

1. Forschungsfrage

Wie beeinflusst die Architektur von TDD-Workflow-Automatisierung die Code-Qualitaet, Effizienz und Disziplin bei AI-gestuetzter Softwareentwicklung?

Konkret: Wenn ein AI-Coding-Agent (Claude Code) eine Programmieraufgabe per Test-Driven Development loest – macht es einen Unterschied, *wie* der TDD-Workflow strukturiert ist? Braucht man ueberhaupt TDD? Reicht ein einfaches “mach TDD”? Oder braucht es spezialisierte Agenten mit strikter Phasentrennung?

2. Experiment-Design

Drei unabhaengige Variablen

A) Workflow-Variante (5 Stufen):

Workflow	Ansatz	TDD?	Mechanismus
v1-oneshot	Direktimplementierung (“Vibe Coding”)	Nein	Anforderungen lesen, Code schreiben, Tests danach
v2-iterative	Plan/Checkliste, schrittweise Umsetzung	Nein	Strukturierte Iteration, Tests danach
v3-basic-tdd	“Verwende TDD” (minimale Anleitung)	Minimal	Agent entscheidet selbst ueber TDD-Prozess
v4-exact-subagents	EXACT-Coding: Jede Phase als eigener Subagent	Strikt	Task(subagent_type: "red/green/refactor")
v5-exact-single-context	EXACT-Coding: Alle Phasen in einer Konversation	Strikt	Skill(skill: "red/green/refactor")

B) Modell-Konfiguration (4 Stufen):

Config	Modell	Thinking
opus	Claude Opus 4.6	Aktiviert (Extended Thinking)
opus-no-thinking	Claude Opus 4.6	Deaktiviert (MAX_THINKING_TOKENS=0)
sonnet	Claude Sonnet 4.5	Aktiviert
sonnet-no-thinking	Claude Sonnet 4.5	Deaktiviert

C) Kata (7 Aufgaben, aufsteigend nach Komplexitaet):

Kata	Komplexitaet	Besonderheit
cerberus-score	Einfach	Scoring-Logik
string-calculator	Einfach	Klassisches TDD-Kata (Roy Osherove)
word-score	Einfach-Mittel	Wortbewertung
pixel-art-scaler	Mittel	Eigenentwicklung – nicht in Trainingsdaten
diamond	Mittel	Algorithmisches Pattern-Generierung
mars-rover	Mittel	Zustandsverwaltung und Befehlsverarbeitung

Kata	Komplexitaet	Besonderheit
game-of-life	Komplex	Infinite Grid, 4 Regeln, Nachbar-Berechnung

Ablauf eines einzelnen Runs

1. Run-Verzeichnis wird erstellt unter `runs/YYYY-MM-DD_HH-MM-SS_kata_workflow_model/`
2. Workflow-Konfiguration (`.claude/rules/`, `.claude/agents/` oder `.claude/commands/`) wird kopiert
3. Abhaengigkeiten werden installiert (TypeScript, Vitest via npm)
4. Claude Code wird automatisch gestartet mit dem Kata-Prompt
5. **Kein menschliches Eingreifen** waehrend des Laufs (HITL deaktiviert)
6. Nach Abschluss: Metriken werden erfasst und Analyse-Report generiert

Metriken

Primaere Qualitaetsmetrik – APP Mass (Code-Komplexitaet):

APP (Absolute Priority Premise) gewichtet Code-Elemente nach Komplexitaet:

Element	Gewicht	Begrueundung
Konstanten	x1	Literale, Strings (niedrig)
Aufrufe	x2	Funktions-/Methodenaufrufe
Bedingungen	x4	if/switch/ternary (Verzweigungslogik)
Schleifen	x5	for/while/map/reduce/forEach
Zuweisungen	x6	Variablenmutationen (hoechste Komplexitaet)

Total Mass = Summe(Anzahl x Gewicht) – niedrigere Mass = einfacherer, wartbarer Code.

Weitere Metriken im Detail:

Duration (Quelle: metrics.json) – Gesamtdauer des Experiment-Runs in Sekunden, vom Start des Claude-Code-Prozesses bis zum Abschluss aller TDD-Phasen. Misst die praktische Effizienz eines Workflows. Niedrigerer Wert = schneller.

Tests (Quelle: Code-Analyse) – Anzahl aktiver Testfaelle (ohne `it.todo()`). Sollte idealerweise der Anzahl der TDD-Zyklen entsprechen.

Tokens (Quelle: experiment-summary) – Gesamtverbrauch an Tokens waehrend des Runs. Misst die “Kosten” eines Workflows. Bei v4-subagents enthaelt dies die Tokens aller Subagenten kumuliert. Niedrigerer Wert = effizienter und kostenguenstiger.

Ctx Util (Quelle: experiment-summary) – Kontextfenster-Auslastung am Ende des Runs in Prozent. Bei v4-subagents bleibt der Hauptkontext niedrig (Subagenten haben eigene Kontexte). Bei v5-single-context akkumulieren Tokens, daher ist hoehere Auslastung zu erwarten. Hohe Auslastung (>50%) kann zu Qualitaetsverlust fuehren.

Cycles (Quelle: experiment-summary) – Anzahl abgeschlossener Red-Green-Refactor-Zyklen. Sollte idealerweise der Testanzahl entsprechen. Weicht die Zahl ab, hat der Workflow Zyklen uebersprungen oder wiederholt.

Refactorings (Quelle: experiment-summary) – Anzahl tatsaechlich durchgefuehrter Refactoring-Verbesserungen. In echtem TDD ist Refactoring nach jedem Green-Schritt *Pflicht* (mindestens eine bewusste Evaluierung). Null Refactorings deutet darauf hin, dass der Refactoring-Schritt uebersprungen oder ignoriert wurde. Hoeherer Wert = bessere TDD-Disziplin.

Pred Accuracy / Prediction Accuracy (Quelle: experiment-summary) – Die Vorhersage-Genauigkeit misst das “TDD Guessing Game”. Bevor ein Test ausgefuehrt wird, muss der Agent vorhersagen: *Was wird passieren?* Wird der Test fehlschlagen? Wenn ja, mit welcher Fehlermeldung? Oder wird er bestehen?

Dies prueft, wie gut das **mentale Modell** des LLMs mit der Realitaet uebereinstimmt. Ein Agent, der korrekt vorhersagt, dass ein Test mit “TypeError: foo is not a function” fehlschlaegt, versteht den aktuellen Code-Zustand. Ein Agent, der falsch vorhersagt, hat den Ueberblick verloren.

Angabe als “korrekte/gesamte Vorhersagen” (z.B. “18/19 = 94%”). Hoeherer Wert = besseres Verstaendnis des Code-Zustands. N/A bedeutet, dass der Workflow keine Vorhersagen verlangt (v1, v2, v3).

Tests Immed / Tests Passed Immediately (Quelle: experiment-summary) – Anzahl der Tests, die sofort in der Red-Phase bestehen, *ohne* dass Green-Code geschrieben werden musste. Dies ist ein direkter Indikator fuer **Ueberimplementierung**: Der Agent hat im vorherigen Green-Schritt mehr Code geschrieben, als der vorherige Test verlangte.

In idealem TDD sollte dieser Wert nahe Null sein – jeder neue Test sollte zunaechst fehlschlagen (Red), bevor minimaler Code geschrieben wird (Green). Ein hoher Wert (z.B. 9 von 10 Tests bestehen sofort) bedeutet, dass der Agent die gesamte Loesung auf einmal geschrieben hat und TDD nur “simuliert”. Niedrigerer Wert = bessere inkrementelle TDD-Disziplin.

Bewertungskriterien

Ein Workflow ist **besser**, wenn er produziert: - Niedrigere Code-Mass (einfacherer Code) - Weniger Token-Verbrauch (effizienter) - Hoehere Vorhersage-Genauigkeit (besseres mentales Modell des Code-Zustands) - Mehr Refactorings (sauberer Code, echte TDD-Disziplin) - Weniger sofort bestandene Tests (echtes inkrementelles TDD, keine Ueberimplementierung) - 100% bestandene Tests (korrekte Implementierung) - Niedrigere Standardabweichung (konsistentere Ergebnisse)

3. Bekannte Einschraenkung: Trainingsdaten-Kontamination

Die klassischen Katas (String Calculator, Game of Life, Diamond, Mars Rover) sind extrem verbreitet in Coding-Dojos, Buechern, Blog-Posts und tausenden GitHub-Repositories. **Claude hat mit hoher Waerscheinlichkeit dutzende Loesungen waehrend des Trainings gesehen.**

Moegliche Auswirkungen: - Kuenstlich niedrige Code-Mass (Modell reproduziert “bekannte gute Loesungen”) - Kuenstlich hohe Vorhersage-Genauigkeit (Modell “weiss” bereits das Ergebnis) - Mehr Ueberimplementierung (Modell weiss, was als naechstes kommt) - Weniger Refactoring-Bedarf (Implementierung ist von Anfang an “richtig”)

Gegenmasssnahme: Der `pixel-art-scaler` wurde speziell fuer dieses Experiment entwickelt und existiert nicht in den Trainingsdaten. Seine Trends stimmen mit den anderen Katas ueberein, ein einzelnes Novel-Kata reicht aber nicht fuer belastbare Aussagen.

4. Workflow-Architektur im Detail

v1-oneshot – Keine Struktur

Agent --> Anforderungen lesen --> Code schreiben --> Tests danach

Der Baseline-Workflow. Keine TDD-Regeln, keine Struktur. Der Agent liest die Anforderungen und implementiert alles in einem Durchgang. Tests werden erst nach der Implementierung basierend auf dem Example Mapping ergaenzt. Repraesentiert “normales Coding” ohne TDD-Disziplin.

v2-iterative – Plan ohne TDD

Agent --> Anforderungen lesen --> Checkliste erstellen --> Schritt fuer Schritt umsetzen --> Tests danach

Strukturierte Iteration mit explizitem Plan, aber ohne Test-First-Ansatz. Die Hypothese war, dass Planung die Code-Qualitaet verbessert. **Die Daten zeigen das Gegenteil** – v2 produziert konsistent den komplexesten Code aller Workflows.

v3-basic-tdd – “Mach einfach TDD”

Agent --> "Verwende TDD" --> Agent entscheidet Vorgehen selbst

Minimale TDD-Anweisungen ohne phasenspezifische Regeln. Die Kontrollgruppe: Reicht es, dem Agenten zu sagen “mach TDD”? In der Praxis: Das Modell schreibt Tests und Code, aber ohne echte Red-Green-Refactor-Disziplin.

Konfiguration: v3-basic-tdd/.claude/rules/experiment-mode.md

v4-exact-subagents – Isolierter Kontext pro Phase

Main Agent

```
|-- Task(test-list) --> eigener Subagent, frischer Kontext
|-- Task(red)        --> eigener Subagent, frischer Kontext
|-- Task(green)      --> eigener Subagent, frischer Kontext
|-- Task(refactor)   --> eigener Subagent, frischer Kontext
```

Jede TDD-Phase laeuft in einem **separaten Agenten** mit eigenem, sauberem Kontext. Der Zustand (existierender Code, aktueller Test) wird explizit per Prompt-Parameter uebergeben. Jeder Agent hat einen spezialisierten System-Prompt.

Vorteile: Strikte Isolation erzwingt Disziplin – der Green-Agent *kann nicht* ueber zukuenftige Tests wissen. Kein Kontext-Rauschen aus vorherigen Phasen.

Nachteile: Agent-Spawning-Overhead (langsam). Zustandsverlust zwischen Phasen moeglich. Ohne Extended Thinking verliert das Modell den Faden.

Konfiguration:

```
v4-exact-subagents/.claude/
|-- agents/test-list.md, red.md, green.md, refactor.md
|-- rules/tdd.md, tdd_with_ts_and_vitest.md, tdd-experiment-mode.md
```

v5-exact-single-context – Geteilter Kontext

Single Agent

```
|-- Skill(/test-list) --> selber Agent, selber Kontext
|-- Skill(/red)        --> selber Agent, selber Kontext
|-- Skill(/green)      --> selber Agent, selber Kontext
|-- Skill(/refactor)   --> selber Agent, selber Kontext
```

Alle Phasen laufen **im selben Agenten** mit durchgehendem Kontext. Skills liefern phasenspezifische Anweisungen, aber der Agent behaelt das volle Gedaechnis ueber alle Phasen.

Vorteile: Kein Zustandsverlust. Kein Agent-Spawning-Overhead. Schneller.

Nachteile: Risiko von Kontext-Verschmutzung. Agent koennte vorgreifen (weniger strikte Isolation). Kontext waechst mit der Zeit.

Konfiguration:

```
v5-exact-single-context/.claude/
|-- commands/test-list.md, red.md, green.md, refactor.md
|-- rules/tdd.md, tdd_with_ts_and_vitest.md, tdd-experiment-mode.md
```

Architektur-Vergleich

Aspekt	v4 (Subagents)	v5 (Single Context)
Kontext	Isoliert pro Phase	Geteilt ueber alle Phasen
Zustandsuebergabe	Explizit per Prompt	Implizit im Gespraechsverlauf
Dauer	10-30 Min (Overhead)	7-11 Min
Zustandsverlust	Ja – Agent “vergisst”	Nein – alles bleibt
Disziplin	Strenger (erzwungene Isolation)	Etwas lockerer (koennte vorgreifen)
Definitionen	agents/*.md	commands/*.md

5. Technische Infrastruktur

Verzeichnisstruktur

```
experiments/
|-- katas/                                Standardisierte Programmieraufgaben
|   |-- string-calculator/prompt.md
|   |-- game-of-life/prompt.md
|   |-- mars-rover/prompt.md
|   |-- diamond/prompt.md
|   |-- pixel-art-scaler/prompt.md
|   |-- cerberus-score/prompt.md
|   +--- word-score/prompt.md
|
|-- workflows/                            Workflow-Varianten
|   |-- v1-oneshot/.claude/rules/
|   |-- v2-iterative/.claude/rules/
|   |-- v3-basic-tdd/.claude/rules/
|   |-- v4-exact-subagents/.claude/{agents,rules}/
|   +--- v5-exact-single-context/.claude/{commands,rules}/
|
|-- runs/                                 Experiment-Ergebnisse
|   +--- YYYY-MM-DD_HH-MM-SS_kata_workflow_model/
|       |-- .claude/                      Workflow-Konfiguration
|       |-- src/                          Generierter Code
|       |-- metrics.json                  Erfasste Metriken
|       |-- analysis-report.md            Analyse-Report
|       +--- prompt.md                    Kata-Prompt
|
|-- docker/                              Containerisierte Umgebung
|-- record-run.sh                         Interaktiv: neuen Run starten
+--- analyze-run.sh                       Runs analysieren/vergleichen
```

Docker-Ausfuehrung (empfohlen)

Experimente laufen in isolierten Docker-Containern fuer Reproduzierbarkeit.

Setup:

```
cd experiments/docker
cp .env.example .env
# ANTHROPIC_API_KEY in .env setzen
echo "USER_ID=$(id -u)" >> .env
```

```
echo "GROUP_ID=$(id -g)" >> .env
docker compose build
```

Interaktiver Run:

```
docker compose run -it --rm experiment bash
./record-run.sh
# Kata und Workflow auswaehlen
```

Batch-Modus (alle Kombinationen automatisch):

```
docker compose --profile batch run --rm batch
```

Container-Details: - Base Image: node:22-slim - Tools: Node.js 22, pnpm, TypeScript 5.3, Vitest 1.0, Claude Code CLI, jq, git - Ressourcen: max 2 CPU-Kerne, max 4GB RAM - Sicherheit: Non-root User (experimenter), Read-only Mounts fuer Katas/Workflows

Volume-Mounts:

Host	Container	Modus
../katas	/home/experimenter/experiments/katas	ro
../workflows	/home/experimenter/experiments/workflows	ro
../runs	/home/experimenter/experiments/runs	rw

Batch-Ablauf pro Run: 1. Isoliertes Run-Verzeichnis erstellen 2. Workflow-Regeln kopieren 3. Abhaengigkeiten installieren 4. Claude mit 30-Minuten-Timeout starten 5. Metriken und Output aufzeichnen

Lokale Ausfuehrung (Alternative)

```
cd experiments
./record-run.sh          # Interaktiv: Kata + Workflow waehlen
./analyze-run.sh --all   # Alle Runs analysieren
```

Katas aktivieren/deaktivieren

Praefix _ deaktiviert ein Kata oder einen Workflow fuer Batch-Runs:

```
mv katas/game-of-life katas/_game-of-life    # Deaktivieren
mv katas/_game-of-life katas/game-of-life     # Wieder aktivieren
```

6. Ergebnisse (235 Runs)

6.1 Code-Komplexitaet (APP Mass)

Die primaere Qualitaetsmetrik. Niedrigerer Wert = einfacherer Code.

opus-no-thinking (groesste Stichprobe: 3-5 Runs pro Zelle):

Kata	v1-oneshot	v2-iterative	v3-basic-tdd	v4-subagents	v5-single-ctx
cerberus-score	26	34	20	39	16
diamond	139	156	119	126	124
game-of-life	180	235	185	200	168
mars-rover	126	138	120	154	120

Kata	v1-oneshot	v2-iterative	v3-basic-tdd	v4-subagents	v5-single-ctx
pixel-art-scaler	58	76	57	59	47
string-calculator	37	56	45	26	42
word-score	50	55	39	48	33

Fett = hoechster (schlechtester) bzw. niedrigster (besten) Wert pro Zeile.

Kernbefunde: - v5 gewinnt 5/7 Katas - v2 ist auf allen 7 Katas am schlechtesten - v4 gewinnt nur beim einfachsten Kata (string-calculator)

opus mit Thinking (1 Run pro Zelle):

Kata	v1	v2	v3	v4	v5
cerberus-score	20	20	20	7	29
diamond	114	122	111	97	99
game-of-life	155	137	140	122	171
mars-rover	123	93	94	88	116
pixel-art-scaler	65	65	52	41	32
string-calculator	29	29	21	21	25
word-score	33	28	28	33	26

Mit Thinking gewinnt v4-subagents 5/7 Katas – eine vollstaendige Umkehrung.

6.2 Thinking-Modus: Der v4-Interaktionseffekt

Das bedeutendste Ergebnis. Opus mit vs. ohne Thinking innerhalb v4:

Kata	v4 opus	v4 opus-nt	Mass-Reduktion
cerberus-score	7	39	-82%
diamond	97	126	-23%
game-of-life	122	200	-39%
mars-rover	88	154	-43%
pixel-art-scaler	41	59	-31%
string-calculator	21	26	-19%
word-score	33	48	-31%

v4 mit Thinking gewinnt **alle 7 Katas** mit 19-82% Mass-Reduktion. Zudem ist v4 mit Thinking auf 5/7 Katas auch **schneller** (weniger vergeudete Zyklen und Retries).

Erklaerung: Die Subagent-Architektur gibt jeder Phase einen isolierten Kontext. Mit Extended Thinking denkt das Modell tief innerhalb dieses fokussierten Kontexts nach – es versteht, was existiert, was minimal ist, was zu aendern ist. Ohne Thinking wird die Isolation zum Nachteil: Das Modell verliert den Ueberblick und ueberimplementiert.

6.3 Geschwindigkeit

Konsistente Reihenfolge ueber alle Katas (schnellstes zuerst):

Rang	Workflow	Typische Dauer
1	v1-oneshot	~1-2 Min

Rang	Workflow	Typische Dauer
2	v2-iterative	~1.5-3 Min
3	v3-basic-tdd	~3-5 Min
4	v5-exact-single-context	~7-11 Min
5	v4-exact-subagents	~10-30 Min

v4-subagents ist **7-14x langsamer** als v1-oneshot. v5 braucht etwa die Haelfte der Zeit von v4. Der Overhead bei v4 entsteht durch das wiederholte Spawnen von Subagenten fuer jede TDD-Phase.

6.4 TDD-Disziplin

Metrik	v3-basic-tdd	v4-subagents	v5-single-ctx
Refactorings (Durchschnitt)	0-1	1-6	1-7
Vorhersage-Tracking	N/A ueberall	58-100%	75-100%
Ueberimplementierung	Hoch	Mittel	Niedrig

v3 macht kein echtes TDD: Null Refactorings, keine Vorhersagen, haeufige Ueberimplementierung. Dem Agenten “mach TDD” zu sagen produziert etwas naeher an “schreibe Tests und Code” ohne Red-Green-Refactor-Rigor.

v5 kontrolliert Ueberimplementierung am besten: Auf pixel-art-scaler hat v3/Sonnet 9 Tests, die sofort bestehen (effektiv null Red-Phasen). v5 haelt dies bei 2-3.

6.5 Modell-Vergleich

Opus produziert einfacheren Code als Sonnet, besonders bei komplexen Katas. Auf game-of-life: Opus Mass 171, Sonnet Mass 268 – Sonnet produziert 57% komplexeren Code.

Extended Thinking hilft Opus: 5/7 Katas zeigen niedrigere Mass mit Thinking auf v5, und 7/7 auf v4.

6.6 Zuverlaessigkeit

Success Rate: 100% ueberall ausser einer Kombination. v5-single-context auf mars-rover mit opus-no-thinking: 75% (3 von 4 Runs). Alle anderen 234 Runs: 100% Erfolg.

6.7 Datenqualitaets-Probleme

Problem	Betroffene Metriken
Clean-Code-Metriken identisch ueber alle Workflows pro Kata	LOC, Functions, LOC/Func
Coverage identisch ueber alle Workflows pro Kata	Statements, Branches
Token-Zaehlung = 0 fuer einige TDD-Runs	Tokens, Ctx Util
Vorhersage-Genauigkeit > 100% in einzelnen Runs	Pred Accuracy

7. Schlussfolgerungen

Gesicherte Erkenntnisse

1. **TDD produziert einfacheren Code** – alle drei TDD-Workflows (v3, v4, v5) produzieren niedrigere Mass als die Non-TDD-Baselines auf den meisten Katas.
2. **Strukturierte Iteration ohne TDD (v2) schadet aktiv** – konsistent der komplexeste Code auf allen 7 Katas. Eine Checkliste ohne Test-First-Disziplin foerdert Over-Engineering.
3. **“Mach einfach TDD” (v3) reicht nicht** – akzeptable Mass-Werte (wahrscheinlich aus Trainingsdaten-Wissen), aber keine echte TDD-Disziplin: keine Refactorings, keine Vorhersagen, haeufige Ueberimplementierung.
4. **Der beste Workflow haengt vom Thinking-Modus ab:**
 - **Mit Thinking:** v4-subagents (isolierter Kontext + tiefes Nachdenken = disziplinierter, einfacher Code)
 - **Ohne Thinking:** v5-single-context (geteilter Kontext kompensiert flacheres Reasoning)
5. **Opus produziert einfacheren Code als Sonnet**, besonders bei komplexen Katas. TDD verkleinert den Abstand.
6. **Extended Thinking hilft Opus** – 5/7 Katas mit niedrigerer Mass auf v5, 7/7 auf v4.

Workflow-Ranking

Ohne Thinking:

Rang	Workflow	Mass	TDD-Disziplin	Dauer
1	v5-exact-single-context	Niedrigste (5/7)	Echt: Refactorings, Vorhersagen	~8-10 Min
2	v3-basic-tdd	2.-3. niedrigste	Schein-TDD	~3-4 Min
3	v4-exact-subagents	Mittelfeld	Echt, aber inkonsistent	~15-30 Min
4	v1-oneshot	Mittelfeld	Keine	~1-2 Min
5	v2-iterative	Hoehste (schlechteste)	Keine	~1.5-3 Min

Mit Thinking:

Rang	Workflow	Mass	TDD-Disziplin	Dauer
1	v4-exact-subagents	Niedrigste (5/7)	Strikteste: hohe Refactorings	~12-16 Min
2	v5-exact-single-context	2. niedrigste	Echt	~7-11 Min
3	v3-basic-tdd	3. niedrigste	Schein-TDD	~2-5 Min
4	v1-oneshot	Mittelfeld	Keine	~1-2 Min
5	v2-iterative	Hoehste (schlechteste)	Keine	~1.5-3 Min

Offene Fragen

1. **Generalisierbarkeit** – klassische Katas sind in Trainingsdaten. Ein Novel-Kata reicht nicht.
2. **Optimales Thinking-Budget** – nur on/off getestet, keine Zwischenstufen.

3. **Skalierung auf komplexe Aufgaben** – game-of-life und mars-rover deuten auf andere Dynamiken bei komplexen Problemen hin.
 4. **Clean-Code-Metriken** – Analyzer scheint fuer LOC, Functions und Coverage defekt zu sein.
-

Anhang: Experiment-Matrix

Kata	v1	v2	v3	v4	v5	Total
cerberus-score	6	6	6	6	6	30
diamond	6	6	6	6	6	30
game-of-life	6	6	6	5	6	29
mars-rover	6	6	6	6	7	31
pixel-art-scaler	7	7	7	7	7	35
string-calculator	8	8	8	8	8	40
word-score	8	8	8	8	8	40
Total	47	47	47	46	48	235